FN-CUSTOS-REDPROOF-001

A CI red-proof record: verdict-consistency ≠ substantive detection ≠ coverage

Prepared for	Tavares Heath — independent reconstruction test (CUSTOS-CORE red-proof)
Reconstruction ID	`FN-b27fe08ed9252198`
Engine version	camsva 0.9.0
Generated	2026-10-13
Status	**FINAL — BOUNDED**

Purpose & boundary

This states, per proposition, what the records **establish**, **contradict**, or **cannot establish**.

STRICTLY NON-DISPOSITIVE. FactNotebook does not re-run the suite and does not judge whether CUSTOS-CORE's controls are adequate; it establishes what one preserved, public red-proof record permits an independent party to conclude, across four propositions that the single verdict RED-CONFIRMED tends to collapse. (1) The verdict is CONSISTENT with its own declared rule (P1, ESTABLISHED) — but rule-consistency is not yet a substantive result. (2) The recorded non-zero outcome is corroborated (P2, ESTABLISHED) across the JUnit, the pytest output, and — independently of the subject's record — GitHub's own job-step outcomes; what is corroborated is that a test FAILED / the outcome was non-zero, not the exact value by an independent channel. (3) An INTENDED admin-auth assertion FAILED with exactly the symptom the injection produces (P3, ESTABLISHED): the one failing test asserts an admin endpoint must require auth and observed it returning 200 — the effect of disabling token verification — with 32 tests passing first. That this was CAUSED by the injection is a strong inference (matching symptom + the subject-declared red-proof premise that the suite is green on clean code), not a byte-level proof: no post-injection diff or before/after run was captured, and FactNotebook does not re-run. (4) That EVERY intended auth-regression test failed — the full class was exercised (the sabotage script's own claim) — is CANNOT ESTABLISH (P4): pytest ran with -x and stopped at the first failure: 33 tests ran (32 passed, 1 failed), so only one relevant FAILING assertion was observed; the rest of the class was not reached. The load-bearing distinction: a verdict consistent with its rule does not establish the substantive proposition it represents, and one assertion failing does not establish coverage of the class. The CANNOT ESTABLISH on P4 is a successful reconstruction of a real coverage boundary, not a defect — addressing it would need re-running THIS sabotage without -x (the subject's proposed third sabotage class probes a DIFFERENT coverage question — believed-untested behaviour — not this class). No Exempt Node: FactNotebook admits no interpretation of its own as an established fact; every observation here is an independently retrieved, hash-verified record. Re-derivability here is INTERNAL determinism: FactNotebook re-ran this reconstruction and the identifier and states reproduced; the frozen observation set — its hash, the selected observation ids and the versioned rules — is set out in the 'Frozen reconstruction manifest' reference below (and, in full, in Annex D of the machine-readable finding), and the public artifacts it points to can be independently re-fetched and re-hashed by anyone. Third-party replay of the reconstruction engine is not offered (the engine is not public), so independent verification is at the observation level, not the engine level.

Provenance & processing

Every observation points to a public, re-walkable artifact. The evidence record, JUnit XML and pytest output were retrieved via the nightly.link public artifact proxy (GitHub's API required authentication); the JUnit and pytest-output SHA-256 were recomputed here and match the record. The workflow, sabotage script and code-at-commit were retrieved from raw.githubusercontent; the sabotage script's SHA-256 matches the record's sabotage_script_sha256. The job-step outcomes were read from GitHub's public Actions API. NB: matching SHA-256 shows CONSISTENCY between the retrieved files and the record's references — not, by itself, an independent attestation that the run was produced as described, nor of the truth of the artifacts' substantive claims; that the run executed and returned non-zero is attested separately by GitHub's own job-step outcomes — GitHub-hosted metadata recorded apart from the subject's evidence artifact, though dependent on the same workflow execution.

Scope & coverage

Records were selected and supplied by Tavares Heath's CUSTOS-CORE 'red-proof' CI (the SUBJECT; public repo ceyptoslim/CUSTOS-CORE, AGPL). The records are the subject's own CI output, published on GitHub; FactNotebook retrieved and SHA-256-verified them itself (it was not handed them). They are NOT an independent-of-subject source; the only separately-hosted corroboration (GitHub's own job-step outcomes) still depends on the same workflow execution. This finding is bounded to that submitted surface; completeness beyond it — and whether an adverse class of observations was withheld — is not established. The instrumentation was selective, not exhaustive: a selective extraction is not an exhaustive observation of the surface.

See Annex C (acquisition / selection boundary) and Annex B (non-admitted candidate register).

1 — Verdict consistency

Proposition	State	Basis
the recorded verdict RED-CONFIRMED follows the declared verdict rule (pytest_exit_code != 0 -> RED-CONFIRMED) applied to the recorded exit code (1)	**ESTABLISHED**	From the evidence record v2 (SHA-verified): pytest_exit_code = 1; rule 'observations.pytest_exit_code != 0 -> RED-CONFIRMED; else FAULTY-GREEN'; verdict = RED-CONFIRMED. 1 != 0 => RED-CONFIRMED. This establishes the verdict is CONSISTENT with its own declared rule — an internal-consistency check on the record. It does NOT by itself establish that the underlying run is faithful (P2) or that the substantive regression was detected (P3)

2 — Recorded outcome ↔ execution evidence

Proposition	State	Basis

the recorded non-zero pytest outcome (a test genuinely failed during the run) is corroborated by the preserved execution reports and GitHub's observed workflow behaviour, independently of the record's verdict field	**ESTABLISHED**	The pytest output (hash-matched) reads '1 failed, 32 passed' and the JUnit XML (hash-matched) carries tests='33', failures='1' — two views of the SAME run, both the subject's artifacts. The INDEPENDENT corroboration is GitHub's own job steps (not the subject's record): step 7 'Enforce red-proof verdict' (which would exit 1 on a zero exit code) succeeded, and step 9 'Red confirmed' (guarded by exit_code != 0) ran rather than being skipped — so exit_code != 0 on GitHub's own record. These step outcomes are GitHub-hosted execution metadata, recorded SEPARATELY from the subject-generated evidence artifact but DEPENDENT on the same workflow execution and the values it passes — a second, differently-hosted view, not a fully independent re-measurement. Scope: what is corroborated is that a test FAILED / the outcome was NON-ZERO; the exact value '1' is the record's JSON field and is not separately re-derived

3 — Substantive detection

Proposition	State	Basis

during run 38446904362 an intended admin-auth assertion failed with exactly the symptom the injected regression produces — an admin endpoint (/v1/audit) no longer requiring authentication	**ESTABLISHED**	The single failing test (pytest runs with -x) is tests/test_admin_auth.py::TestAdminEndpointsRequireAuth::test_audit_endpoints_reject_missing_token: 'AssertionError: /v1/audit must require auth, got 200' (assert 200 == 401) — an admin endpoint no longer requiring authentication, which is exactly the effect of disabling token verification. The 32 preceding passing tests support that the suite was operational, but do NOT independently exclude every environment-specific cause of this single failure. The causal chain is kept honest, at three levels: (i) the script was DESIGNED to disable auth — established (it does `sed 's/return verify_token(credentials)/return None/' main.py`, and that target exists at main.py@6ed3cff, lines 122 & 137); (ii) the script RAN — corroborated by GitHub's job step 5 'Apply known-bad sabotage patch' (success), independently of the subject's record; (iii) that this modification CAUSED this specific failure is a strong INFERENCE (matching symptom + the subject-declared red-proof premise that the suite is green on clean code), NOT a byte-level demonstration — no post-injection diff or before/after execution was captured here, and FactNotebook does not re-run. What is ESTABLISHED is the scoped claim in the proposition; full causal proof would be more demanding

Limitations

P4-regression-class-covered — *every intended 401/auth regression test failed under the sabotage — the full admin-auth regression class was exercised and detected (the sabotage script's own claim: 'Every 401/auth regression test MUST fail')* → **CANNOT ESTABLISH** (no_channel). CANNOT ESTABLISH — the workflow runs pytest with -x, so execution STOPPED at the first failing assertion (JUnit: 33 testcases = 32 passed + 1 failed;

pytest output: 'stopping after 1 failures'). Exactly ONE intended auth assertion was observed to fail; the remaining auth-regression tests in the class were not run, so the record does not permit establishing that the whole class failed. Reason class: coverage / sufficiency. Addressing it would need a re-run of THIS sabotage (001) WITHOUT -x, so the whole class executes. (The subject's proposed third sabotage class probes a DIFFERENT coverage question — whether believed-untested behaviour is covered — not the coverage of this regression class.) This is the observation-set limit the subject declared

Separation of authority

Observation instruments determine what they observe and propose. Admission determines what may enter the reconstruction record. FactNotebook determines what that admitted record permits us to establish about the proposition. An instrument may search, select, transform or directly acquire observations according to its own declared procedure; those operations remain attributable properties of the instrument, not hidden properties of the reconstruction engine.

recorded behaviour → **observation** (attributed to its instrument) · declared risk / rule classifications → **the evaluator's / authority's policy** (never engine facts) · the consequential judgment → **the accountable authority**, outside this reconstruction.

Re-derivability

Two levels are preserved. **The finding** re-derives: re-running the reconstruction against the admitted observations reproduces the reconstruction identifier `FN-b27fe08ed9252198` and the states in this document — with no re-run of the probabilistic extraction. The admitted set equals the full candidate set in this reconstruction; no detailed per-candidate admission register was produced. The acquisition boundary is documented in Annex C; a separately reviewed admission pass would produce a traceable register. *Version the instrument · trace the extraction · freeze the admitted record · replay the reconstruction.*

References

Each observation points to a citable source a third party can re-walk; the status records how it was verified.

CUSTOS-CORE red-proof evidence record v2 (001-weaken-admin-auth) — run_id 38446904362, commit 6ed3cff1935f…, injected_regression_id 001-weaken-admin-auth, sabotage_script_sha256 82a0329d…; pytest_exit_code 1; verdict RED-CONFIRMED under rule v2; ci_job_outcome 'success' = the test STEP's outcome (it ends in `tail` under `set +e`), not a pass · *PRIMARY — independently retrieved (nightly.link); JUnit & pytest-output SHA-256 match the record* · `https://github.com/ceyptoslim/CUSTOS-CORE/actions/runs/38446904362`

Execution evidence (JUnit XML + pytest output) — 33 tests, 32 passed, 1 failed (pytest -x); the one failure = tests/test_admin_auth.py::TestAdminEndpointsRequireAuth::test_audit_endpoints_reject_missing_token — '/v1/audit must require auth, got 200' (assert 200 == 401) · *PRIMARY — retrieved via nightly.link; SHA-256 68086270…a3b73e / 45b1c5c3…c1dec match the record* · `artifact red-proof-records-001-weaken-admin-auth (public run page)`

Workflow + sabotage + code — ci-red-proof.yml runs `pytest tests/ -q --no-header -x …` and derives the verdict from the exit code; 001-weaken-admin-auth.sh does `sed 's/return verify_token(credentials)/return None/' main.py`; main.py@6ed3cff has that line at 122 & 137 · *PRIMARY — raw.githubusercontent; sabotage SHA-256 matches the record; sed target present at the commit* · `https://github.com/ceyptoslim/CUSTOS-CORE/blob/main/.github/workflows/ci-red-proof.yml`

GitHub-hosted job-step outcomes — job 'Sabotage 001-weaken-admin-auth must turn tests RED': step 7 'Enforce red-proof verdict' success (would exit 1 on a zero exit code), step 9 'Red confirmed' (guarded by exit_code != 0) ran => exit != 0 · *SEPARATELY-HOSTED — GitHub Actions public API; recorded apart from the subject's evidence artifact but dependent on the same workflow execution* · `https://api.github.com/repos/ceyptoslim/CUSTOS-CORE/actions/runs/38446904362/jobs`

Frozen reconstruction manifest (the inputs a replay fixes) — reconstruction_id FN-b27fe08ed9252198 · observation_model FN-CUSTOS-REDPROOF-001 hash sha256:4eb29176a732927375f60a0d66384fcd20677b2412caf37b4b761ca6ac4d6f05 · observation_set hash sha256:b78ab1ac6f251e7c54eddfa349e213e26e774ef11f10975fcd56b7ae2366cb4c (3 observations: o_8c4efc2f451e68c2, o_8da7d6d3d689dc8b, o_c2d7ecf96f1c3cad) · rule versions fn-reconstruction-engine-v1 / fn-rule-pattern-v1 / fn-composition-v1 / fn-temporal-v1 / fn-canon-v1. Internal determinism: re-running against these frozen inputs reproduces the identifier and the four states. The engine is not public, so independent verification is at the OBSERVATION level — the public artifacts in the references above can be re-fetched and re-hashed by anyone · *FROZEN — mirrors Annex D binding; lets a reader see exactly what a replay holds fixed* · `Annex D (binding)`

Annex B — Non-admitted candidate register

Every candidate was admitted: none rejected, none left unresolved.

Annex C — Acquisition / selection boundary

Records were selected and supplied by Tavares Heath's CUSTOS-CORE 'red-proof' CI (the SUBJECT; public repo ceyptoslim/CUSTOS-CORE, AGPL). The records are the subject's own CI output, published on GitHub; FactNotebook retrieved and SHA-256-verified them itself (it was not handed them). They are NOT an independent-of-subject source; the only separately-hosted corroboration (GitHub's own job-step outcomes) still depends on the same workflow execution. This finding is bounded to that submitted surface; completeness beyond it — and whether an adverse class of observations was withheld — is not established. The instrumentation was selective, not exhaustive: a selective extraction is not an exhaustive observation of the surface.

Surface constituted / submitted by	Tavares Heath's CUSTOS-CORE 'red-proof' CI (the SUBJECT; public repo ceyptoslim/CUSTOS-CORE, AGPL). The records are the subject's own CI output, published on GitHub; FactNotebook retrieved and SHA-256-verified them itself (it was not handed them). They are NOT an independent-of-subject source; the only separately-hosted corroboration (GitHub's own job-step outcomes) still depends on the same workflow execution
Submission form	one public CI run (38446904362): evidence record v2, JUnit XML, pytest output, workflow, sabotage script, code at the commit — published publicly, FN-retrieved
Candidate extraction instrument	the subject's CI (pytest under a known-bad sabotage patch) — FactNotebook re-reads the preserved artifacts, it does NOT re-run the suite
Admission authority / mechanism	FactNotebook (proposer). Three steps are kept distinct, and integrity is NOT treated as sufficiency: (1) ACQUISITION + INTEGRITY — the machine artifacts were independently retrieved and SHA-256 verified: the retrieved artifacts match the recorded cryptographic digests; their substantive claims remain subject to independent evaluation; (2) ADMISSION as attributed observations — they are direct, structured records, so no interpretive admission step is needed (no fuzzy candidate to adjudicate); (3) RELEVANCE + SUFFICIENCY per proposition — appraised in the reconstruction itself, never inferred from integrity: an intact, authentically-retrieved artifact can still be insufficient for a given proposition (P4 is CANNOT ESTABLISH precisely because the record, though intact, cannot carry the coverage claim). The one reasoned link (the failing assertion realises the sabotage's declared intent, P3) is grounded in the assertion's own message. Not an independent audit
Reconstruction engine	camsva 0.9.0
Surface type	submitted
Acquisition mode	selective
Known limitations	Bounded to one run (38446904362, sabotage 001). The evidence record is produced by the subject's own CI, but it was independently retrieved and its JUnit / pytest-output SHA-256 match it, and GitHub's own job-step outcomes corroborate exit != 0 independently of it. The suite was NOT re-run here. Crucially, pytest ran with -x: only the FIRST failing assertion was recorded, so coverage of the full regression class is not observable from this run (P4). This establishes what the preserved record permits — not that CUSTOS-CORE's controls are adequate.

Annex D — Policy & engine binding

Engine camsva 0.9.0 · Reconstruction ID `FN-b27fe08ed9252198`

FactNotebook reconstructs what can be established from the surviving records — including what cannot be — and keeps the establishment separate from the judgment applied to it.